JS tools

A small suite of personal CLI tools that share a common look and feel — colored output, aligned status lines, meaningful exit codes, and `-h` help on every command. Cohesive enough to feel like one program even though each tool is a standalone script.

Design docs: `docs/ARCHITECTURE.md` (the repo map) and `docs/command-surface-contract.md` (the emit-once `describe` contract + the effect model, with diagrams). House rules for editing are in `AGENTS.md`.

Platform: macOS only. The crypt tools rely on `/usr/bin/security` (Keychain), `osascript` (passphrase dialogs), and `open -W`. The `tools watch` agent uses `launchctl`. None of this has Linux/WSL equivalents in-tree.

Requirements:

- `bash` ≥ 4 (Homebrew: `brew install bash`) — macOS ships 3.2, which is missing associative arrays used by `tools doctor`.
- `zsh` ≥ 5 — for the completion file and `dns-test`.
- `age`, `git`, `rsync` — `brew install age git rsync`.
- An age-compatible identity (SSH ed25519 is fine).
- `node` ≥ 20 — only for the Node-based tools (`doc-to-pdf`, `site manage`, `site compare`). Run `npm ci` once to fetch their pinned deps and the JSON Schema validator used by `tools check`.
- `shellcheck` + `bats-core` — only to run `tools check` (the CI suite) locally. The macOS system `/usr/bin/expect` runs the real-PTY coverage for `site manage`.

What's in the box

```
tools/
  bin/                    # public command surface
    # Core
    tools       # Umbrella command for the personal CLI toolchain.
    # Cryptography
    encrypt     # Age-encrypt files to your public key; remove originals.
    decrypt     # Age-decrypt .age files with your private key; restore originals.
    open-age    # Decrypt a .age file to a temp file and open it in the default app.
```

```
    # Vault
    inbox       # Quick-capture a note into the vault inbox.
    vault       # Operations on the vault git repo.
    # Backup
    backup      # Mirror the items in config/backup.sh into $BACKUPS_HOME.
    # Diagnostics
    dns-test    # Compare DNS resolver latency across paths.
    # Drift guards
    ts-acl      # Fetch the live Tailscale ACL policy and diff it against the vault mirror.
    cf-dns      # Fetch the live Cloudflare DNS records and diff them against the vault mirror.
    adguard     # Fetch the live AdGuard Home DNS rewrites and diff them against the vault mirror.
    nginx       # Fetch the live Nginx Proxy Manager proxy hosts and diff them against the vault
mirror.
    # Integrations
    hq          # Sync vault frontmatter to Severino HQ, plus routine HQ deployment ops.
    site        # Public jseverino.com Astro site workflow.
    brand       # Render Joe's brand kits via the branding-engine.
    # Authoring
    remember    # Write a Claude memory file + MEMORY.md index entry in one shot.
    doc-to-pdf  # Render a Markdown file (with Mermaid) to PDF via local Chromium, offline.
    diagram     # Render Mermaid .mmd sources to neighboring PNG files.
  .github/              # CI workflows and repository automation
  archive/              # retired scripts kept for reference
  bench/                # measured claims asserted in CI
  completions/          # generated zsh completion
  config/               # tracked defaults and user-local templates
  docs/                 # architecture and contract explanations
  lib/                  # shared helpers and tool-specific implementation
  schemas/              # machine-enforced cross-tool contracts
  tests/                # hermetic Bats coverage
```

Layout rules, so the repo stays navigable as it grows:

- `bin/` **is the public surface.** Exactly one executable per tool, no support files. `tools install`, the zsh completions, and CI all discover tools by iterating `bin/*`, so adding a tool means dropping one file there.
- `lib/` **splits shared from tool-specific.** Helpers used by every tool (`common.sh`, `init.sh`, `key.sh`) sit flat; anything that belongs to one tool lives under `lib/<tool>/`, so a tool's footprint is obvious and removable.

- **Tools that outgrow a script become their own project.** `site compare` launches the sitedrift viewer from its own checkout (override the entry point with `SITEDRIFT_ENTRY`) rather than vendoring a copy here.

Install

```
brew install bash zsh age git rsync
git clone <this-repo> ~/path/to/tools
export TOOLS_HOME=~/path/to/tools   # add this to ~/.zshrc

cp config/backup.sh.example config/backup.sh   # then edit for your files
tools install                                  # symlinks into ~/.local/bin
tools doctor                                   # verify env, deps, symlinks
tools key cache                                # one-time passphrase cache
```

`tools install` is idempotent — re-run after pulling or adding a new tool to refresh the symlinks. Override the install target with `TOOLS_INSTALL_DIR=/somewhere/else tools install`. Make sure that directory is on `$PATH`.

Layout env vars

Each tool resolves paths in two tiers:

1. **Tool-specific env var** (e.g. `AGE_PUBKEY`, `VAULT`) — explicit per-invocation override; takes precedence.
2. **Layout var** (e.g. `KEYS_HOME`, `NOTES_HOME`) — required, read from your shell environment; tools error out with a clear message if not set.

Example `.zshrc` block — adapt to your own paths:

```
export TOOLS_HOME="$HOME/code/tools"          # this repo
export NOTES_HOME="$HOME/code/notes"          # vault repo (with .git/)
export KEYS_HOME="$HOME/code/keys"            # age public/private key pair
export BACKUPS_HOME="$HOME/code/backups"      # mirror destination
```

`config/vault.sh` and `config/crypt.sh` synthesize the derived paths (`VAULT`, `INBOX_DIR`, `AGE_PUBKEY`, `AGE_KEY`) from these. Change a layout var and every tool follows.

If you reorganize, edit only the layout vars in `~/.zshrc` — never the tracked configs.

Zsh completion

Add this to `~/.zshrc` **before** `compinit`:

```
fpath=("$TOOLS_HOME/completions" $fpath)
```

Then restart your shell. The completion file is generated from every tool's `--describe` contract, so commands, flags, choices, and positional arguments stay aligned with help automatically. Regenerate it with `tools generate`.

Conventions

- **Output**: header → status lines → optional summary → trailing newline.
- **Status lines**: a colored verb (`encrypted`, `captured`, `pulled`, etc.) in a fixed-width column, then the path or detail, then optional dim context.
- **Exit codes**: `0` success or only-skips, `1` at least one failure, `2` usage error (bad flag, missing args).
- **Flags**: `-h` / `--help` always works. `-f` / `--force` opts into clobber where it makes sense.

Tools

CLI Reference

This reference is generated from the same `--describe` contracts as `-h`, zsh completion, and the TUI. Run `tools generate readme` after changing a command surface.

`tools`

Umbrella command for the personal CLI toolchain.

Invocation	Arguments / options	Effect	Summary
`tools status`	`--json`	`read + network`	One-screen health check across vault, inbox, backup, keys

Invocation	Arguments / options	Effect	Summary
`tools doctor`	`--all` `--live` `--json`	`read + network`	Verify environment, deps, and installed symlinks
`tools check`	`--no-bench`	`local_write`	Run the full CI suite locally: lint, tests, bench
`tools new <name>`	`<name>` `--drift` `--verify`	`local_write`	Scaffold a new tool in bin/ with the house conventions
`tools install`	—	`local_write`	Create symlinks in $INSTALL_DIR for every tool
`tools key [cache	forget	status	test]`
`tools watch [enable	disable	status	run-now]`
`tools describe [tool] [command]`	`[tool]` `[command]` `--pretty` `--repos` `--tui`	`read`	Emit the command surface of every tool as one JSON document (the emit-once contract)
`tools tui`	`--repos`	`read + interactive`	Open the full-screen command-surface explorer (shorthand for 'describe --tui')
`tools generate [all	completions	readme]`	`[all|completions|readme]` `--check`		

`tools describe` details

Examples

```
tools describe hq restart  # one command's contract — effect, args, examples
tools tui  # interactive explorer over the whole toolchain (alias of describe --tui)
```

`tools tui` details

The human tier of the emit-once contract: a two-pane explorer over every tool and command, with / to filter, e to expand the selected command's full prose + examples, and Enter to copy a ready-to-paste invocation.

`encrypt`

Age-encrypt files to your public key; remove originals.

Encrypts files to your default age public key (and any extras passed with -k). The original file is removed on success unless -c is given.

Usage: `encrypt <file>...`

Argument	Description
`-c, --copy`	Keep the original file (encrypt a copy)
`-f, --force`	Overwrite existing .age output files
`-k, --key <PATH>`	Add another public key as a recipient (repeatable)
`<file>...`	File(s) to encrypt

Effect: `local_write`

Examples

```
encrypt notes.md  # original removed
encrypt -c ~/.ssh/id_ed25519  # original kept (backup pattern)
encrypt -k ~/keys/coworker.pub notes.md
encrypt -k a.pub -k b.pub *.md
encrypt -f *.txt
encrypt -- -starts-with-dash.md
```

`decrypt`

Age-decrypt .age files with your private key; restore originals.

Decrypts .age files using your default age private key. If the key is an SSH key with a passphrase, decrypt unlocks it transparently using the passphrase cached via 'tools key cache' — one prompt up front, silent forever after. With no cached passphrase, it prompts (terminal or osascript dialog, whichever is appropriate).

Usage: `decrypt <file>...`

Argument	Description
`-f, --force`	Overwrite existing decrypted output files
`-k, --key <PATH>`	Add another identity to try (repeatable). Bypasses the cached passphrase / unlock path for that key.
`-p, --stdout`	Write decrypted bytes to stdout, not a file. Silent on success; status/errors go to stderr (e.g. decrypt -p secret.age | less).
`--no-cache`	Don't use the cached passphrase even if one exists. Lets age prompt directly (terminal only).
`<file>...`	The .age file(s) to decrypt

Effect: `local_write + interactive`

Examples

```
decrypt notes.md.age
decrypt -k ~/keys/oldkey notes.md.age
decrypt -f *.age
decrypt -p secret.age | less
```

`open-age`

Decrypt a .age file to a temp file and open it in the default app.

Decrypts a .age file to a temporary file under $TMPDIR (mode 600, owner-only), opens it in the default macOS app for the underlying extension via 'open -W', then deletes the temporary file when the app finishes with it.

The plaintext never lands in the source directory or anywhere persistent. $TMPDIR is per-user and cleared by macOS on reboot.

Set it as the default opener for .age files with 'duti -s <bundle.id.of.this.script> .age all', or wrap this script in a tiny .app bundle (Automator: Run Shell Script).

Usage: `open-age <file>`

Argument	Description
`--no-cache`	Skip the cached passphrase, let age prompt directly

Argument	Description
`<file>`	The .age file to open

Effect: `local_write + interactive`

`inbox`

Quick-capture a note into the vault inbox.

Capture a quick note into your vault inbox folder. The filename is “YYYY-MM-DD HHMMSS .md” and the body is the captured text.

Usage: `inbox [text]...`

Argument	Description
`-e, --edit`	Open the captured note in $EDITOR after writing. With no text/stdin, creates an empty note, opens the editor, then renames the file from its first non-empty content line on save.
`[text]...`	Note text (joined). Omitted when piping stdin.

Effect: `vault_write`

Examples

```
inbox "remember to update the homelab certs"
pbpaste | inbox
echo "lookup later" | inbox
inbox -e  # blank note, opens editor
inbox -e "draft: post-mortem template"  # seed + open editor
```

`vault`

Operations on the vault git repo.

Invocation	Arguments / options	Effect	Summary
`vault sync`	—	`remote_write + network`	Pull and push the vault repo

Invocation	Arguments / options	Effect	Summary
`vault status`	—	`read + network`	Working tree, inbox count, remote sync state, and whether doc metadata changed since the last hq sync
`vault inbox`	—	`read`	List notes currently in the inbox

`backup`

Mirror the items in config/backup.sh into $BACKUPS_HOME.

Mirror each tracked file into $BACKUPS_HOME using the mapping in config/backup.sh. Uses rsync, so permissions, xattrs, and timestamps are preserved and unchanged files are skipped at the byte level.

If $BACKUPS_HOME is a git repo, an auto-commit is made after the copy so each backup run leaves a point-in-time entry in the local history.

Add or remove items by editing tools/config/backup.sh.

Usage: `backup`

Argument	Description
`-n, --dry-run`	Show what would be copied; do not write
`--no-commit`	Skip the auto-commit step

Effect: `local_write`

`dns-test`

Compare DNS resolver latency across paths.

Measures DNS latency across System DNS, a LAN AdGuard resolver, Cloudflare 1.1.1.1, and Cloudflare DoH. Reports avg/min/p50/p95/max and the delta of each path against a chosen baseline.

The AdGuard IP defaults to an example LAN address; set ADGUARD_IP in your environment (or pass -a) to point at your own resolver.

Usage: `dns-test`

Argument	Description
`-d <domain>`	Domain to query (default: google.com)
`-n <runs>`	Queries per path (default: 20)
`-a <ip>`	AdGuard LAN IP (default: 192.168.1.233, env ADGUARD_IP)
`-b <label>`	Path label to use as delta baseline (default: “AdGuard LAN”)

Effect: `read + network`

`ts-acl`

Fetch the live Tailscale ACL policy and diff it against the vault mirror.

Invocation	Arguments / options	Effect	Summary
`ts-acl show`	—	`read + network`	Fetch and print the live state (normalized, sorted JSON)
`ts-acl diff`	—	`read + network`	Diff live vs the vault mirror; exit 1 on drift
`ts-acl pull`	—	`vault_write + network`	Regenerate the vault mirror block from live (accept drift)

`cf-dns`

Fetch the live Cloudflare DNS records and diff them against the vault mirror.

Records are normalized to type/name/content/proxied (+ priority for MX);

Cloudflare-internal fields (id, timestamps, meta) are dropped.

Invocation	Arguments / options	Effect	Summary
`cf-dns show`	—	`read + network`	Fetch and print the live state (normalized, sorted JSON)
`cf-dns diff`	—	`read + network`	Diff live vs the vault mirror; exit 1 on drift

Invocation	Arguments / options	Effect	Summary
`cf-dns pull`	—	`vault_write + network`	Regenerate the vault mirror block from live (accept drift)

`adguard`

Fetch the live AdGuard Home DNS rewrites and diff them against the vault mirror.

Rewrites are normalized to domain/answer, sorted by domain.

Invocation	Arguments / options	Effect	Summary
`adguard show`	—	`read + network`	Fetch and print the live state (normalized, sorted JSON)
`adguard diff`	—	`read + network`	Diff live vs the vault mirror; exit 1 on drift
`adguard pull`	—	`vault_write + network`	Regenerate the vault mirror block from live (accept drift)

`nginx`

Fetch the live Nginx Proxy Manager proxy hosts and diff them against the vault mirror.

Proxy hosts are normalized to domain_names / forward_scheme / forward_host /

forward_port / enabled, sorted by domain.

Invocation	Arguments / options	Effect	Summary
`nginx show`	—	`read + network`	Fetch and print the live state (normalized, sorted JSON)
`nginx diff`	—	`read + network`	Diff live vs the vault mirror; exit 1 on drift
`nginx pull`	—	`vault_write + network`	Regenerate the vault mirror block from live (accept drift)

`hq`

Sync vault frontmatter to Severino HQ, plus routine HQ deployment ops.

Severino HQ glue. Reads YAML frontmatter from the vault and upserts the HQ docs index. Also wraps the routine ssh + docker compose calls for managing the HQ deployment.

Invocation	Arguments / options	Effect	Summary
`hq manifest`	—	`read`	Print the manifest JSON (frontmatter from every doc) to stdout — for inspection or piping elsewhere
`hq sync`	`--prune` `--no-update`	`remote_write + network`	Pipe the manifest into HQ's import_docs_manifest; upserts by doc_id, safe to re-run
`hq doctor`	—	`read`	Report vault docs missing or with invalid frontmatter, and whether HQ's last-synced manifest is stale
`hq schema`	`--check`	`local_write`	Regenerate HQ's docs_index/schema.json from the installed MCP (the canonical frontmatter contract)
`hq validate`	—	`read + network`	Report HQ registry entries (Projects/Assets) that no vault doc references. Read-only
`hq create <project`	asset> `	`<project|asset> <slug>`	`remote_write + network`
`hq deploy`	—	`deploy + network`	Fallback: re-pull the latest scanned GHCR image and restart the container (CI's deploy step)
`hq ship`	`-m, --message <TEXT>`	`deploy + network`	Commit + push a small HQ change. The push IS the deploy: it triggers the gated pipeline (build → scan → deploy on green)
`hq logs`	`-f, --follow` `--tail <N>`	`read + network`	Show app container logs (default tail 50)
`hq restart`	—	`deploy + network`	`docker compose restart app` — no rebuild, no migrations. For env changes or stuck state
`hq open`	—	`read`	Open $HQ_URL in the browser

Invocation	Arguments / options	Effect	Summary
`hq shell`	—	`remote_write + network + interactive`	ssh -t into the HQ Django shell (poke the ORM)
`hq superuser`	—	`remote_write + network + interactive`	ssh -t into HQ and run createsuperuser interactively
`hq export [year] [md`	json]`	`[year] [md|json]`	`local_write + network`

`hq create` details

Examples

```
hq create project my-site --name "My Site" --category cloudflare --status published --url
https://example.com
hq create asset example-com --name "example.com" --category domain
```

Examples

```
hq sync  # most common — push vault → HQ
hq manifest | jq '.[] | .doc_id'
hq doctor  # find docs missing frontmatter
hq export 2026  # download year-summary-2026.md
```

`site`

Public jseverino.com Astro site workflow.

Invocation	Arguments / options	Effect	Summary
`site status`	—	`read`	Show repo location, git state, and build-output state
`site sync`	—	`local_write`	Sync public pages/writeups from the vault into the site repo
`site check`	—	`local_write`	Run Astro diagnostics

Invocation	Arguments / options	Effect	Summary
`site contrast`	—	`read`	Compute WCAG ratios for every text/background pair in base.css
`site parity`	—	`read`	Assert vault Frontmatter Schema, Zod, and MCP agree on writeup fields
`site build`	—	`local_write`	Run the full Astro build
`site publish`	`--no-push`	`deploy + network`	Ship every change: gate published writeups, hq sync, build + audits, auto-commit, push, then verify each affected writeup live
`site sign-security`	—	`local_write + interactive`	Clear-sign public/.well-known/security.txt with the security@ key
`site check-security`	—	`read`	Verify signature, required fields, Expires, and WKD file
`site scaffold-primer`	Delegated: the site repo's `npm run scaffold:primer` — run it with --help for flags	`vault_write`	Scaffold a new 04 Reference/ primer with slim frontmatter
`site scaffold-field`	Delegated: the site repo's `npm run scaffold:writeup-field` — run it with --help for flags	`local_write`	Patch every layer needed for a new writeup field (dry-run by default)
`site draft-alt`	Delegated: the site repo's `npm run draft:cover-alt` — run it with --help for flags	`vault_write + network`	Use the Claude API to draft cover_alt from a writeup's cover image
`site publish-all`	`--no-push`	`deploy + network`	Alias of publish
`site publish-writeup <slug>`	`<slug>`	`deploy + network`	Full publish flow with the named writeup highlighted; the all-published gate runs once first
`site validate <slug>`	`<slug> --draft`	`read`	Run the writeup publish gate standalone — report only, no build/commit

Invocation	Arguments / options	Effect	Summary
`site tech [query]`	`[query]`	`read`	List technology slugs from the vault catalog, filtered when a query is given
`site featured [slug] [target]`	`[slug]` `[target]`	`vault_write`	Show the home-page featured order, or move one writeup and renumber automatically
`site manage`	—	`vault_write + interactive`	Interactive manager: every writeup on one screen — reorder, feature, publish. Nothing written until you save
`site verify <slug>`	`<slug>`	`read + network`	Post-publish live check: page status, OG image, tag pages, and home placement
`site diagnose`	Delegated: the site repo's `npm run diagnose` — run it with --help for flags (–fast, --json)	`read`	The collect-all gate: run every audit and report all failures in one pass
`site release <version>`	`<version>` `--ship`	`deploy + network`	Bump package.json, run publish:check, commit + signed tag, push, create the GitHub release
`site test`	`--visual` `--ui` `--update`	`local_write`	Run the Playwright end-to-end suite
`site doctor`	—	`read + network`	Pre-flight health check: CLI/npm drift, vault-mcp install, security, contrast, parity, type check, audit. No build
`site reinstall-mcp`	—	`local_write`	Reinstall the severino-vault-mcp package from source
`site new-writeup <slug>`	`<slug>`	`vault_write`	Scaffold a new vault writeup folder from the template (starts published: false)
`site seo <page>`	`<page>` `-r, --result`	`read`	Preview the Google-style search result snippet for a built page

Invocation	Arguments / options	Effect	Summary
`site dev`	`--drafts`	`local_write + interactive`	Start the local Astro dev server
`site open`	—	`read`	Open the local dev URL in the browser

`site compare [path]`	`[path]`
	`--dev <URL>`
	`--live <URL>`
	`--mobile`
	`--desktop`
	`--compact`
	`--expanded`
	`--link-scroll`
	`--no-link-scroll`
	`--scroll-mode <exact|ratio>`
	`--mirror-links`
	`--no-mirror-links`
	`--split <PERCENT>`
	`--swap`
	`--solo`
	`--split-view`
	`--overlay`
	`--overlay-diff`
	`--focus <dev|live>`
	`--notes`
	`--note <TEXT>`
	`--notes-out <FILE>`
	`--clear-notes`
	`--brand <NAME>`
	`--http`
	`--no-open`
		`read + interactive`	Open a resizable dev-vs-live browser comparison

`site og`	—	`local_write`	Regenerate the Open Graph social card (public/assets/og/)

`site publish` details

Runs the whole vault-to-live flow: gate every published writeup, hq sync, build + audits, auto-commit the synced snapshot, push (Cloudflare Pages rebuilds), then verify each affected writeup live. Only the generated snapshot is committed; drafts stay in the vault and are never pushed.

`site publish-writeup` details

The same gate as `site publish` validates every published writeup once before any sync, build, commit, or push. Requires the severino-vault-mcp console script on PATH.

`site tech` details

Catalog source: 06 Pages/_technology-groups.md. Featured slugs surface in the home-page cloud.

`site featured` details

No args prints the order the home page renders; a slug + target moves that writeup and renumbers 1…N (inserting one sets featured: true). The new order ships on the next `site publish`.

Examples

```
site featured my-writeup top  # move to slot 1
site featured my-writeup off  # unfeature; others close the gap
```

`site manage` details

Full-screen manager (keys shown in-app): reorder featured, feature/unfeature, publish/unpublish across every writeup; saving submits one transactional plan. Ship with `site publish`.

`site verify` details

Checks the live site: /portfolio// is 200, the og:image resolves, every tag page lists it, and the home page shows it when featured. Run ~30s after publishing (Cloudflare rebuild).

`site new-writeup` details

Creates 05 Writeups//index.md (published: false) and an images/ folder; stays out of the build until you set published: true.

`site seo` details

Reads dist.nosync — run `site build` first if the page is missing.

Examples

```
site seo portfolio
```

```
site seo --result architecting-a-custom-detection-engine
```

`site compare` details

Both panes share a route and a draggable divider; the viewer documents its own keys. Review notes live in a shared file the viewer polls, so a teammate or an AI session can leave notes live — set SITE_COMPARE_VAULT (defaults to the vault 00 Inbox) for the drawer's Send-to-vault button.

`brand`

Render Joe's brand kits via the branding-engine.

Kits land in $BRAND_HOME/kits. The engine lives in $ENGINE_HOME and comes in as a dependency of the brand repo; see its README for all flags.

Invocation	Arguments / options	Effect	Summary
`brand build`	—	`local_write`	Rebuild every kit from brand/ + the social cards
`brand kit <slug> <hex> <initials> [wordmark]`	`<slug>` `<hex>` `<initials>` `[wordmark]` `--font <FONT>` `--only <LIST>` `--out <DIR>`	`local_write`	Render a one-off kit into severino-brand/kits/
`brand status`	—	`read`	Show engine + brand locations and git state

`remember`

Write a Claude memory file + MEMORY.md index entry in one shot.

Compress the two-step "write file + edit MEMORY.md" loop into one command.

Body content is read from stdin unless --body is given. Frontmatter is generated from the args.

Usage: `remember <type> <slug> <title>`

Argument	Description
`-d, --description <STR>`	Frontmatter description (default: title)
`-k, --hook <STR>`	MEMORY.md one-liner hook (default: title)
`-b, --body <STR>`	Body text in lieu of stdin
`-f, --body-file <FILE>`	Body text from file
`-F, --force`	Overwrite an existing memory (update in place)
`--dir <PATH>`	Memory dir. Default: CLAUDE_MEMORY_DIR, else CLAUDE_PROJECT_DIR, else $PWD — encoded to ~/.claude/projects//memory. No flag needed inside a Claude project.
`--list`	Show the MEMORY.md index
`--forget <SLUG>`	Delete the memory and its index entry
`<type>`	user | feedback | project | reference
`<slug>`	kebab- or snake-case identifier (becomes 'name:' in frontmatter; underscored in filename)
`<title>`	text shown as the link in MEMORY.md

Effect: `local_write`

Examples

```
echo "rule body here" | remember feedback use-rg-and-fd "Use rg and fd"
remember feedback use-rg-and-fd "Use rg and fd" -F < updated-body  # update
remember --list
remember --forget use-rg-and-fd
```

`doc-to-pdf`

Render a Markdown file (with Mermaid) to PDF via local Chromium, offline.

Produces a branded document using the Joe Severino kit and embedded Inter variable font. The kit resolves from DOCTOPDF_BRAND_KIT, then BRAND_HOME, then CODE_HOME/Assets/severino-brand.

Markdown image references are consumed unchanged. Rare inline Mermaid fences are rendered through the diagram tool, so both commands share one Mermaid implementation and brand configuration.

Usage: `doc-to-pdf <input.md> [output.pdf]`

Argument	Description
`<input.md>`	Markdown file to render.
`[output.pdf]`	Output path (default: .pdf beside the input).

Effect: `local_write`

`diagram`

Render Mermaid .mmd sources to neighboring PNG files.

Each path may be an .mmd file or a directory. Directories render their top-level .mmd files.

Rendering uses Mermaid CLI 11.15.0 with Joe Severino brand tokens, PNG output, 1100px width, 3x scale, and a white background. Set DIAGRAM_BRAND_KIT to override the kit directory.

Author flow diagrams with flowchart TB. Contract diagrams may use HTML labels for dense multiline nodes; site diagrams stay on SVG labels when their nodes are single-line.

Usage: `diagram <path>...`

Argument	Description
`<path>...`	Mermaid source file or directory

Effect: `local_write`

Examples

```
diagram docs/diagrams/
diagram docs/diagrams/architecture.mmd
```

tools

Umbrella command for managing the suite itself. Its generated command reference is above; this section explains the higher-level behavior.

`tools status` is the daily health check; `tools doctor` is the new-machine smoke test. `tools doctor --all` is the cross-system rollup: after the local checks it runs every system's own gate — `hq doctor` (frontmatter validity + vault→HQ sync freshness), `hq schema --check` (contract parity), `site doctor` (seams, install drift, security, types, audits) — each timed, with a failing gate's output tail inlined, and one verdict / exit code at the end. `--live` also runs the drift guards (`cf-dns` / `adguard` / `ts-acl` diff): live API reads that need network and the age key. The gate registry lives in `lib/doctor.sh`; a gate's only contract is "exit 0 when healthy", so adding one is one line. Both commands take `--json` for machine-readable output (useful for agents and cron). `TOOLS_INSTALL_DIR` overrides the install target.

`tools check` is the same command CI runs — the definition of "passing" lives in one place. It discovers scripts by shebang (`bash -n`, `zsh -n`, `node --check`), runs shellcheck over the tracked shell sources, the bats test suite in `tests/`, validates every describe document against `schemas/cordon-v4.json`, checks generated surfaces, and runs the bench assertions in `bench/`. `tools new <name>` drops a canonical skeleton in `bin/` — init.sh sourcing, usage block, arg loop, correct exit codes — and prints the follow-ups. `tools new <name> --drift` scaffolds a drift-guard tool instead (the `ts-acl` / `cf-dns` / `adguard` / `nginx` shape): `show` / `diff` / `pull` with `get_token` / `fetch_live` / `normalize` / `vault_block` already wired, plus a matching `config/<name>.sh` — fill in the TODOs (endpoint, creds key, jq projection) and seed the vault block with `<name> pull`.

Add `--verify` to either scaffold command to regenerate derived surfaces and run `tools check --no-bench`.

A successful `pull` writes the regenerated block through the `severino-vault-mcp` console script (`severino-vault-mcp update-mirror-block <vault-relative-path> --heading <h> --touch-reviewed`, JSON on stdin) — never a raw file write. The replacement is scoped to the mirror's own heading section (a second mirror in the same doc can never be matched or clobbered), validated to parse as JSON, and lands in **one** atomic write that also stamps the doc's `last_reviewed` to today — a pull is a review. When the MCP CLI isn't installed or the doc lives outside `$NOTES_HOME`, `pull` falls back to an awk rewrite with the same section scoping, staged in the doc's own directory, and stamps `last_reviewed` via `touch-reviewed`, best-effort. `diff` reads with the same scoping and fails loudly when the section has no block (instead of comparing against nothing). Either way the loop is `<tool> diff` → reconcile the prose → `<tool> pull` → `hq sync`.

tools describe — the command-surface contract

Every tool emits its command surface as one structured JSON document conforming to **Cordon**, the language-agnostic command-surface contract — this toolchain is Cordon's reference Bash emitter. Three consumers render from that single source: an AI session reads it, the `--tui` explorer builds a picker from it, and CI guards diff it. This is **emit-once, render-many** — a tool declares its surface once in a `describe_spec()` (the `desc_*` DSL in `lib/describe.sh`), and *both* `-h` / `--help` and `--describe` are derived from it, so the human help and the machine JSON can never drift (there is no prose to parse).

```
encrypt --describe            # one tool's contract (compact JSON)
tools describe                # federated: every tool, one document
tools describe --pretty       # indented, for reading
tools describe encrypt        # just one tool
tools describe hq restart     # just one command — the token-minimal AI path
tools describe --repos        # also fold in sibling repos (severino-vault-mcp)
tools describe --tui          # full-screen explorer: browse + copy invocations
```

Every command (and leaf tool) also declares its **effect** — a blast-radius class (`read | local_write | vault_write | remote_write | deploy`) plus `network` / `interactive` tags — via one `desc_effect` line. It's the signal an agent risk-gates on before running a command (a `deploy` vs a `read`), shown in the focused `-h`, colored in `--tui`, and carried in the JSON.

`--tui` is the human tier of the contract: a two-pane explorer (tools | the selected tool's commands/options/args) over the same federated document. `Tab` / `←→` switch panes, `↑/↓` move, `/` filters tools *and* commands across the whole toolchain, `Enter` copies a ready-to-paste invocation, `q` quits. Aggregate only — a single tool stays the clean `<tool> -h`. It shares the `site manage` look via the common TUI library (`lib/tui.mjs`).

The contract — a superset of what `severino-vault-mcp describe` emits:

```
{ "ok": true, "schema_version": 4, "name": "encrypt",
  "description": "…", "group": "Cryptography", "order": 20,
  "effect": "local_write",
  "global_options": [ { "name": "--copy", "positional": false,
                        "required": false, "help": "…",
                        "flags": ["-c","--copy"], "takes_value": false } ],
  "positionals":   [ { "name": "file", "positional": true,
                       "required": true, "help": "…" } ],
```

```
  "commands":      [ { "name": "…", "summary": "…", "args": [ … ],
                       "effect": "deploy", "network": true } ] }
```

Output is byte-deterministic (no timestamps), so a guard can diff it across runs. `tools doctor` gates that every tool answers `--describe` with a valid contract — and because both views render from the one `describe_spec`, that gate plus the round-trip test in `tests/describe.bats` keep the whole suite self-describing as it grows. `lib/describe.sh` runs under both bash and zsh, so the lone zsh tool (`dns-test`) self-describes from the same engine.

For the full design — the DSL, the `schema_version 4` shape, the effect model, scoped lookup, federation, and the diagrams — see `docs/command-surface-contract.md`. The repo map is `docs/ARCHITECTURE.md`.

tools key — passphrase cache

If your `$AGE_KEY` is a passphrase-protected SSH key (the default ed25519 with `ssh-keygen`'s prompt), `decrypt` and `open-age` would otherwise prompt for the passphrase on every call. Cache it in the login Keychain once with `tools key cache`; use the other generated `tools key` actions to inspect, test, or clear it.

After `tools key cache`, every `decrypt`, `open-age`, and Finder integration runs silently — no prompts, no terminal popups.

How it works. Storage is `/usr/bin/security` under service `age-key-passphrase`, account `$USER` — same mechanism as `git-credential-osxkeychain`. When `decrypt` runs:

1. Fetch cached passphrase from Keychain (silent if cached).
2. Copy `$AGE_KEY` to a fresh `$TMPDIR` file (mode 600).
3. Run `ssh-keygen -p -P <passphrase>` to strip the passphrase from the copy (canonical OpenSSH unlock — no `expect`, no pseudo-TTYs).
4. Pass the unlocked copy to `age -i` for the actual decryption.
5. Delete the unlocked copy on exit (trap covers crashes too).

Threat model. Cached with `-A` (any app running as you can read it) — same effective protection as the SSH key file's mode 600. Not a security upgrade over the file; a UX upgrade for non-interactive contexts. The key file's passphrase still protects it in iCloud, git history, and backups.

Bypass the cache for a single call: `decrypt --no-cache file.age`.

tools watch — opt-in auto-sync

Off by default. When enabled, launchd fires `vault sync` every `TOOLS_WATCH_INTERVAL` seconds (default 900 = 15 min). Output appended to `tools/.logs/vault-sync.log` (gitignored). Override the launchd label via `TOOLS_WATCH_LABEL` (default `com.tools.vault-sync`).

encrypt / decrypt

Wrappers around age for locking and unlocking files with an SSH ed25519 key. See `SECURITY.md` for the threat model — in particular, what encryption-at-rest does and does *not* protect against.

`encrypt` removes the plaintext after successful encryption unless `-c` is given. `decrypt` always leaves the `.age` file in place. Use `decrypt -p` to view or pipe a secret without leaving plaintext on disk: `decrypt -p secret.age | less`.

Examples

```
encrypt notes.md secrets.txt              # original removed
encrypt -c ~/.ssh/id_ed25519              # original kept
encrypt -k ~/keys/coworker.pub notes.md   # default + coworker
decrypt notes.md.age
decrypt -k ~/keys/oldkey notes.md.age     # add a second identity to try
decrypt -p config.json.age | jq .          # view without touching disk
decrypt --no-cache notes.md.age            # ignore cache, age prompts directly
```

open-age

Decrypt-and-open for `.age` files. Pulls plaintext into `$TMPDIR` (mode 600), opens it in the OS-default app for the underlying extension via `open -W`, then removes the temp when the app finishes with it. Plaintext never lands in the source directory or anywhere persistent.

Set as the macOS default opener for `.age` to make double-clicking “just work”:

```
brew install duti
duti -s com.apple.Terminal .age all   # or your own .app bundle id
```

For multi-window editors (VS Code, Sublime), `open -W` returns when the *editor* exits, not when you close the window. For those flows, prefer `decrypt -p file.age | <viewer>` (no temp file at all) or

use a single-document app as the default for `.age`.

inbox

Quick-capture a note into the vault inbox folder.

Filename is `YYYY-MM-DD HHMMSS <first words>.md` so notes sort chronologically and have a readable name. Body is the captured text, prefixed with a small `created:` frontmatter block. `--edit` opens the captured note and renames a blank capture from its first non-empty line on save.

Examples

```
inbox "remember to update the homelab certs"
pbpaste | inbox                          # capture clipboard
echo "$URL" | inbox                      # capture a URL
inbox -e                                 # blank note, opens $EDITOR
inbox -e "draft: post-mortem template"   # seed + open $EDITOR
```

Defaults in `config/vault.sh`. Override with `VAULT` or `INBOX_DIR`.

vault

Operations on the vault repo.

Defaults in `config/vault.sh`.

hq

Glue between an Obsidian vault and **Severino HQ** — a small private Django ops app (sources at `joeseverino/severino-hq`) that I use as a documentation + projects + assets index. `hq` reads YAML frontmatter from every `.md` under `01 Projects/`, `02 Infrastructure/`, `03 Runbooks/` and upserts the HQ docs index, and wraps the routine `ssh + docker compose` calls for managing the deployment.

`hq <subcommand> --help` for full flag lists. Subcommands that touch HQ records (`sync`, `create`) are idempotent — re-running upserts by key.

`config/hq.sh` requires three env vars in your `~/.zshrc`:

```
export HQ_SSH_HOST=hq-host                      # entry in ~/.ssh/config
export HQ_REMOTE_PATH=/opt/apps/severino-hq     # path on the server
export HQ_URL=https://hq.example.com            # URL where HQ is served
```

`sync` pipes the manifest through `ssh "$HQ_SSH_HOST"` and runs `docker compose exec -T app python manage.py import_docs_manifest -` on the target container. A successful sync also records the shipped manifest's hash (plus the vault HEAD and the exact dirs) at `~/.local/state/severino-tools/hq-sync.json`, so `vault status` and `hq doctor` can report *exactly* when doc metadata has changed since the last sync — the hash covers only the frontmatter manifest HQ imports, so prose-only edits never flag. `deploy` / `logs` / `restart` wrap the equivalent `docker compose` calls; they assume `severino-hq`'s repo layout but are easy to adapt if you fork.

Example workflow

A typical day touches three surfaces — vault docs, HQ records, and the running container — without leaving the terminal:

```
# Edit a runbook in Obsidian, bump last_reviewed in the frontmatter, save.
hq sync                             # push the change to HQ's docs index

# Add a new project + supporting asset.
hq create project my-tool \
    --name "My Tool" --category automation --status active \
    --repo https://github.com/me/my-tool
hq create asset my-tool-com --name "my-tool.com" --category domain

# Ship a code change to the Django app.
cd ~/Projects/severino-hq && git push origin main
hq deploy                           # pulls + rebuilds on $HQ_SSH_HOST

# Tail logs after deploy. Restart if you only edited the .env on the server.
hq logs --tail 100
hq restart
```

Every subcommand is idempotent (`sync`, `create`, `deploy`) or read-only (`logs`, `manifest`, `doctor`), so the whole flow is safe to retry.

Adding a new doc

1. Copy `00 Templates/Runbook.md` (or `Infra Doc.md` / `Decision Record.md`) in the vault.
2. Fill in `doc_id`, `title`, `system`, the rest of the frontmatter.
3. `hq sync`.

Editing an existing doc

Change its frontmatter (e.g. bump `last_reviewed`, flip `status` to `deprecated`), save, `hq sync`. The doc_id is the upsert key — no duplicates.

site

Publishing workflow for the public `jseverino.com` Astro site (sources at `joeseverino/jseverino.com`). The Obsidian vault is the source of truth: `site` syncs the public pages and writeups out of the vault, builds the static output with Astro, and ships it to Cloudflare Pages.

The writeup lifecycle: `site new-writeup <slug>` → write in Obsidian → `site dev --drafts` to preview → flip `published: true` → `site publish`. The publish command needs no slug — it gates every published writeup, builds, ships, and verifies the affected pages on the live site itself. (`site publish-all` remains as an alias; `site publish-writeup <slug>` runs the same batch gate once and highlights the named slug.)

Publishing fails closed when `severino-vault-mcp` is missing, stale, or cannot run. The writeup gate is never skipped.

`site manage` initializes through one `severino-vault-mcp writeup-dashboard` call, which shares a single writeup, catalog, and vault snapshot between the list and publish-readiness results. Saving sends one JSON plan to `apply-writeup-plan`; scalar edits and the complete featured order are staged and committed as one locked transaction, with rollback if any replacement fails.

`site manage`: publishing control plane

The TUI is an operator surface over the publishing system, not a second implementation of its rules:

- **Read model:** one dashboard call returns writeup summaries and validation from the same cached vault snapshot, so displayed state and gate results cannot disagree because of separate scans.
- **Staging model:** feature order, publish flips, and editable scalar fields remain in memory until save. The UI can show the complete pending change without partially mutating the vault.
- **Commit boundary:** save submits one structured plan to the MCP. Schema validation, sequential featured ordering, format-preserving YAML updates, locking, and rollback stay in the owner service.
- **Operational surface:** the Site tab composes server state, git state, build artifacts, security checks, live reachability, and the existing doctor/diagnose/build/test/publish commands instead of duplicating them.
- **Terminal contract:** raw input is decoded as a stream, including split escape sequences and bracketed paste. Editing is grapheme-aware, long fields scroll horizontally with the cursor, and frames use Unicode terminal-cell widths plus vertical viewports for small or resized panes.
- **Verification:** hermetic replay tests cover model transitions and MCP payloads; a real pseudo-terminal test runs the interactive branch at a small window size and verifies paste, arrow decoding, alternate-screen behavior, and terminal-mode restoration.

The failure boundary is deliberate: loading fails closed if the dashboard is unavailable, saving is all-or-nothing through the MCP, and external commands temporarily leave the alternate screen so their native output and exit status remain visible. Quitting with staged changes requires an explicit save or discard decision.

Before any MCP-backed site command runs, `site` compares the installed package fingerprint with the local MCP source checkout. Interactive commands offer to reinstall a missing, legacy, or stale package immediately. Noninteractive commands fail with `site reinstall-mcp` instead of forwarding an incompatible subcommand and exposing raw parser output.

`hq manifest` and `hq sync` delegate frontmatter parsing to `severino-vault-mcp hq-manifest`. This keeps vault indexing, MCP reads, and HQ imports on one parser and rejects duplicate `doc_id` values before syncing.

Set `SITE_MCP_AUTO_REINSTALL=1` for trusted local automation that should repair install drift without prompting.

`site seo <url|path|slug>` reads the built Astro HTML from `dist.nosync` and renders a Google-style search result preview with canonical, title, description, robots, and Open Graph checks. Pass a domain, page slug, writeup slug, or absolute path; add `--result` for only the search-result mockup; run `site build` first if the page has not been built locally.

`site compare [path]` opens local development and the live site in a labeled, resizable split view. It proxies both sides through localhost so the production site's frame protections remain intact. It includes stable linked scrolling, mirrored internal navigation, fixed mobile-width panes, collapsible chrome, reload-free swapping, a one-pane Solo mode, Google previews, and URL-backed review notes. Exact pixel-locked scrolling is the default; ratio mode remains available when page heights differ. Every UI state has a CLI flag: `--mobile`, `--compact`, `--link-scroll`, `--scroll-mode`, `--mirror-links`, `--split`, `--swap`, `--solo`, `--focus`, `--notes`, and repeatable `--note`. AI agents should run `site compare [path] --no-open`, then open the printed localhost URL with their browser tool.

Example review launch:

```
site compare /portfolio/ --mobile --compact --mirror-links --link-scroll \
  --notes --note "Check card spacing" --note "Confirm mobile menu parity"
```

The viewer serves trusted local HTTPS at `https://compare.homelab:4178` using the existing Local PKI certificate generated by `cert-gen compare.homelab`. Map `compare.homelab` to `127.0.0.1` locally. Safari is the default; set `SITE_COMPARE_BROWSER` to override it.

The viewer itself is **sitedrift**, a separate project — `site compare` is just the launcher. It looks for the checkout at `~/Documents/Code/Projects/sitedrift/` and `SITEDRIFT_ENTRY` overrides the entry point.

`site publish` is the everyday path: edit a writeup in Obsidian, run it, and the synced snapshot is auto-committed and pushed when content changed — Cloudflare rebuilds within ~30s and the command then verifies each affected writeup on the live site (page status, og:image, tag pages, home placement). If the sync produces no content diff, the command skips commit, push, and verify. The commit message is built from the diff: it names each slug as published (new), edited, or removed. `--no-push` stops after the local build when you want to review the diff first. `site <subcommand> --help` for flag details.

Layout resolves from env vars, all with defaults:

```
export CODE_HOME="$HOME/Documents/Code"             # defaults shown
export SITE_HOME="$CODE_HOME/Projects/jseverino.com"
export NOTES_HOME="$CODE_HOME/Severino Labs"        # vault root
```

`config/site.sh` (copy from `site.sh.example`) is sourced last for further overrides — `SITE_DEV_HOST`, `SITE_DEV_PORT`.

backup

Mirror tracked files into `$BACKUPS_HOME` using the source/destination pairs listed in `config/backup.sh`. Uses `rsync -a` so permissions, xattrs, and timestamps are preserved and unchanged files are skipped at the byte level. Directory destinations are mirrored with `--delete`, so they always match the source contents exactly.

If `$BACKUPS_HOME` is a git repo, each run that actually changes a file auto-commits with a timestamped message — gives you a local-only point-in-time history without timestamped folders.

Configuring the backup set

`config/backup.sh` is gitignored so your personal list stays out of the repo. Copy the template and edit:

```
cp config/backup.sh.example config/backup.sh
$EDITOR config/backup.sh
```

Each entry is `"<source><TAB><dest under $BACKUPS_HOME>"`. Use a real tab between the two fields. Bash's `$'\t'` makes the tab explicit:

```
BACKUP_ITEMS=(
    "$HOME/.zshrc"$'\t'"dotfiles/zshrc"
    "$HOME/.gitconfig"$'\t'"dotfiles/gitconfig"
)
```

dns-test

Compare DNS resolver latency across a few paths — System DNS, a LAN AdGuard resolver, Cloudflare 1.1.1.1, and Cloudflare DoH. Reports avg/min/p50/p95/max plus the delta against a baseline.

Adding a path is one line in the `paths=( ... )` table inside the script — `"Label|sampler|arg"` where the sampler is `sample_dig` or `sample_doh`. Adding DoT is a matter of writing a `sample_dot` that shells out to `kdig +tls`.

ts-acl

Fetch the live Tailscale ACL policy and diff it against the copy stored in the vault, so policy drift gets caught instead of silently going stale.

Auth credentials live age-encrypted at `$TS_ACL_CREDS` (default `$KEYS_HOME/tailscale/ts-oauth.env.age`) as an env file. Use either a plain API access token, or — preferred, read-only — an OAuth client:

```
# either
TS_API_TOKEN=tskey-api-...
# or
TS_OAUTH_CLIENT_ID=k...
TS_OAUTH_CLIENT_SECRET=tskey-client-...
```

An OAuth client scoped to `acl:read` is least-privilege and does not expire; an API access token is full-account and expires in 90 days (rotate before then).

`ts-acl` streams it via `decrypt -p` (no plaintext on disk), exchanges it for a short-lived access token, then reads `GET /api/v2/tailnet/-/acl`. `diff` pulls the fenced ```json block from `$TS_ACL_VAULT_DOC`. Needs `curl` and `jq`.

Setup: create either an API access token or (preferred) an OAuth client scoped to `acl:read` in the Tailscale admin console, then encrypt the env file to `$TS_ACL_CREDS`.

cf-dns

The DNS sibling of `ts-acl`: fetch the live Cloudflare DNS records and diff them against a mirror stored in the vault, so zone drift gets caught instead of silently going stale.

The mirror is a fenced ```json block under `$CF_DNS_VAULT_HEADING` in `$CF_DNS_VAULT_DOC` (the prose tables in that doc are for humans; the block is the diff target). Records are normalized to `type` / `name` / `content` / `proxied` — plus `priority` for `MX` — and Cloudflare-internal fields (id, timestamps, meta) are dropped, so the diff is stable. `diff` re-sorts both sides; `pull` rewrites the block in place. The accept-drift loop is: `cf-dns diff` → reconcile the prose tables → `cf-dns pull` → `hq sync`.

Auth is a Cloudflare API token age-encrypted at `$CF_DNS_CREDS` (default `$KEYS_HOME/cloudflare/cf-dns.env.age`) as an env file:

```
CF_API_TOKEN=...
```

Scope it to `Zone.DNS:Read` (plus `Zone:Read` so `cf-dns` can resolve the zone id from `$CF_ZONE`; pin `$CF_ZONE_ID` to drop that). `cf-dns` streams the token via `decrypt -p` (no plaintext on disk), then reads `GET /zones/{id}/dns_records`. Needs `curl` and `jq`.

Setup: create the scoped token at Cloudflare → My Profile → API Tokens, then `encrypt cf-dns.env` and move the `.age` to `$CF_DNS_CREDS`. Seed the mirror with `cf-dns pull`.

adguard

The homelab-DNS sibling of `cf-dns`: fetch the live AdGuard Home DNS rewrites and diff them against a mirror in the vault, so the rewrite set that routes `*.homelab` and the Tailscale-only `*.jseverino.com` services can't drift unnoticed.

Reads `GET $ADGUARD_URL/control/rewrite/list` and normalizes each entry to `domain` / `answer`, sorted by domain. The mirror is the fenced ```json block under `$ADGUARD_VAULT_HEADING` in `$ADGUARD_VAULT_DOC`; the prose rewrite tables in that doc stay for humans. Same accept-drift loop as `cf-dns`: `adguard diff` → reconcile the tables → `adguard pull` → `hq sync`.

Auth is the AdGuard web-UI login (HTTP basic auth), age-encrypted at `$ADGUARD_CREDS` (default `$KEYS_HOME/adguard/adguard.env.age`):

```
ADGUARD_USER=...
ADGUARD_PASS=...
```

`$ADGUARD_URL` defaults to `http://192.168.1.233:3001` (reachable over LAN or Tailscale). `adguard` streams the creds via `decrypt -p` (no plaintext on disk). Needs `curl` and `jq`.

Setup: `encrypt adguard.env` and move the `.age` to `$ADGUARD_CREDS`, then seed the mirror with `adguard pull`. This tool was scaffolded with `tools new adguard --drift` (see below).

nginx

The reverse-proxy sibling of `cf-dns` / `adguard`: fetch the live Nginx Proxy Manager proxy hosts and diff them against a mirror in the vault, so the host → upstream map (`hq.jseverino.com`, `adguard.homelab`, …) can't drift unnoticed.

Reads `GET $NGINX_URL/nginx/proxy-hosts` and normalizes each host to `domain_names` / `forward_scheme` / `forward_host` / `forward_port` / `enabled`, sorted by domain. The mirror is the

fenced ```json block under `$NGINX_VAULT_HEADING` in `$NGINX_VAULT_DOC`; the prose table in that doc stays for humans. Same accept-drift loop: `nginx diff` → reconcile the table → `nginx pull` → `hq sync`.

NPM has no long-lived API tokens, so `nginx` exchanges the web-UI login for a short-lived Bearer per call (`POST $NGINX_URL/tokens`). The login is age-encrypted at `$NGINX_CREDS` (default `$KEYS_HOME/nginx/nginx.env.age`):

```
NGINX_EMAIL=...
NGINX_PASSWORD=...
```

`$NGINX_URL` defaults to `http://192.168.1.233:81/api` (reachable over LAN or Tailscale). `nginx` streams the creds via `decrypt -p` (no plaintext on disk). Needs `curl` and `jq`.

Setup: `encrypt nginx.env` and move the `.age` to `$NGINX_CREDS`, then seed the mirror with `nginx pull`. This tool was scaffolded with `tools new nginx --drift` (see below).

remember

Write a Claude memory file and its `MEMORY.md` index entry in one shot, instead of the two-step "create the file, then hand-edit the index" loop.

`<type>` is one of `user | feedback | project | reference`. Body comes from stdin (or `--body` / `--body-file`); frontmatter is generated from the args. `-F` / `--force` overwrites an existing memory (and refreshes its index line instead of duplicating it). The memory dir is auto-resolved — `$CLAUDE_MEMORY_DIR`, else `$CLAUDE_PROJECT_DIR`, else `$PWD`, encoded to `~/.claude/projects/<enc>/memory` — so no `--dir` is needed inside a Claude project; pass `--dir` only to override.

```
echo "rule body here" | remember feedback use-rg-and-fd "Use rg and fd"
```

Why it's worth it (measured). One `remember` call vs the manual Read-index → Write-file → Edit-index → verify loop an agent runs by hand, in bytes (a ~4:1 token proxy, cold-write model). `remember` stays flat because it never reads the index; the manual cost grows with `MEMORY.md`:

```
index   NEW(B)   OLD(B)   ratio
   10      722     3302    4.5x
   30      722     5162    7.1x
```

```
100      722    11675   16.1x
300      722    30875   42.7x
```

Reproduce with `bench/remember-token-bench.sh` (self-contained; asserts `remember` is cheaper and runs in CI). The floor is ~1.5× — if the index is already in the agent's context and it skips the verify read — so the win is real even in the best case for the manual flow, and compounds as memories accumulate.

doc-to-pdf

Render a Markdown file to PDF with fully-rendered Mermaid diagrams — entirely offline. Markdown via `markdown-it`, Mermaid from the locally-pinned bundle, and the system Chrome in headless mode as the print engine. No LaTeX toolchain, no Puppeteer download, no network.

Output defaults to the input path with a `.pdf` extension. Falls back to Edge or Chromium if Chrome isn't installed; `CHROME_PATH` overrides the browser. Deps are pinned in the repo-root `package.json` — run `npm ci` once before first use.

Archived

Retired scripts live in `archive/`. They are not on `$PATH` (not in the install manifest) and are kept only for reference or occasional manual use.

- `wp-static` — mirrors a WordPress site into a static directory for Cloudflare Pages / Netlify (wraps `wget --mirror` with WP-aware defaults and a post-processor that fixes shortlinks and strips `?ver=` cache-busters). Used during the WordPress → static migration; kept for re-mirroring the legacy site. Its `wp-static.sh.example` config and `wp-static-postprocess.py` live alongside it in `archive/`.
- `grep-vs-rg.sh` — one-off benchmark comparing `grep` vs `ripgrep` on a directory tree (uses `hyperfine` if present).

Finder integration (optional)

The intended pattern is to wrap `encrypt` / `open-age` / `decrypt` in small Automator workflows so you can right-click → encrypt or double-click a `.age` file:

1. **Automator → New → Quick Action** (for the right-click menu) or **Application** (for double-click default opener).
2. Workflow receives **files or folders** in **Finder**.
3. Add a **Run Shell Script** action, shell `/bin/zsh`, pass input as **arguments**:

```
# Quick Actions run with a minimal env — re-export your layout vars
# here, or source ~/.zshrc with care.
export TOOLS_HOME="$HOME/code/tools"
export KEYS_HOME="$HOME/code/keys"
export PATH="$HOME/.local/bin:/opt/homebrew/bin:/usr/local/bin:$PATH"

for f in "$@"; do
    "$TOOLS_HOME/bin/encrypt" "$f" >/dev/null 2>&1 || \
        osascript -e "display notification \"failed: $f\" with title \"Encrypt\""
done
osascript -e 'display notification "done" with title "Encrypt"'
```

4. Save. The Quick Action shows up in Finder's right-click menu under *Quick Actions*. Bind a keyboard shortcut in **System Settings → Keyboard → Keyboard Shortcuts → Services → Files and Folders**.
5. For double-click on `.age`: save the workflow as an **Application** instead, then `duti -s <bundle-id> .age all` (Automator apps get a bundle id like `com.apple.automator.<app-name>`).

The `.app` / `.workflow` bundles themselves are intentionally not tracked in this repo — they're macOS binary plists with localization data and personal paths baked in. Build your own from the snippet above.

Related: vault pre-commit hook

If you use `git-crypt` in your vault repo, a pre-commit hook that runs `git-crypt status -f` catches accidentally-unencrypted files before they hit history. That hook lives in the vault repo itself, not here. After cloning the vault:

```
git config core.hooksPath .githooks
```

Contributing

See CONTRIBUTING.md.

License

MIT — see LICENSE.